

DOKUMEN TEKNIS PROYEK AKHIR

Pembangunan Sistem Operasi Minimalis Berbasis Linux

- **Mata Kuliah:** Sistem Operasi
 - **Program Studi:** Teknik Informatika
 - **Durasi Proyek:** 4-5 Minggu
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

DAFTAR ISI

1. [Pendahuluan](#)
 2. [Tujuan Pembelajaran](#)
 3. [Spesifikasi Sistem](#)
 4. [Arsitektur Sistem](#)
 5. [Tahapan Implementasi](#)
 6. [Panduan Teknis Detail](#)
 7. [Testing dan Debugging](#)
 8. [Troubleshooting](#)
 9. [Kriteria Penilaian](#)
 10. [Referensi](#)
-

1. PENDAHULUAN

1.1 Latar Belakang

Proyek ini dirancang untuk memberikan pengalaman praktis dalam membangun sistem operasi dari tingkat paling fundamental. Mahasiswa akan memahami bagaimana kernel, userspace, dan komponen sistem operasi lainnya bekerja sama membentuk sistem yang fungsional.

1.2 Ruang Lingkup Proyek

Mahasiswa akan membangun sistem operasi minimalis yang mencakup:

- Kernel Linux yang dikonfigurasi dan dikompilasi sendiri
- Root filesystem dengan utilities dasar
- System initialization process
- Custom applications untuk demonstrasi konsep OS
- Bootloader configuration
- Documentation lengkap

1.3 Deliverables

1. Source code lengkap dalam repository Git
2. Image sistem operasi yang bootable

3. Dokumentasi teknis komprehensif
 4. Video demo sistem berjalan
 5. Presentasi final
-

2. TUJUAN PEMBELAJARAN

2.1 Pemahaman Konsep

Setelah menyelesaikan proyek, mahasiswa diharapkan memahami:

Kernel Space:

- Kernel configuration dan compilation process
- Device drivers dan hardware abstraction
- System calls interface
- Kernel modules dan dynamic loading

Process Management:

- Process creation (fork, exec)
- Process scheduling algorithms
- Inter-process communication (IPC)
- Signal handling
- Process states dan transitions

Memory Management:

- Virtual memory concepts
- Paging dan segmentation
- Memory allocation strategies
- Swap space management
- Memory protection mechanisms

File System:

- Virtual File System (VFS) layer
- Inode dan dentry structures
- File operations dan permissions
- Different filesystem types (ext4, tmpfs, procfs, sysfs)
- Mount points dan filesystem hierarchy

I/O Management:

- Character dan block devices
- Buffer cache
- Device drivers interface
- Interrupt handling

2.2 Keterampilan Teknis

- Linux kernel configuration dan compilation
 - Cross-compilation untuk embedded systems
 - System programming dengan C
 - Shell scripting untuk automation
 - Debugging tools (gdb, strace, ltrace)
 - Performance profiling dan optimization
 - Version control dengan Git
-

3. SPESIFIKASI SISTEM

3.1 Spesifikasi Minimum

Target Platform:

- Architecture: x86_64 (primary), ARM (optional)
- RAM: 256 MB minimum
- Storage: 100 MB untuk minimal system
- Boot method: BIOS/UEFI, USB, atau ISO

Core Components:

- Linux Kernel 5.15+ atau 6.1+
- Busybox 1.35+
- Basic init system
- Essential utilities (sh, ls, ps, top, mount, etc.)

3.2 Spesifikasi Development Environment

Host System Requirements:

- Linux distribution (Ubuntu 20.04+, Debian 11+, atau Arch Linux)
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 20 GB free space
- Internet connection untuk download dependencies

Required Software:

```
# Compiler dan build tools
gcc (version 9+)
make (version 4.2+)
binutils
bison
flex

# Libraries
glibc-dev / musl-dev
libncurses-dev
libssl-dev
libelf-dev
```

```
# Utilities
git
wget/curl
cpio
qemu-system
```

3.3 Fokus Sistem (Pilih Salah Satu)

Mahasiswa harus memilih fokus sistem mereka:

Option A: Educational OS

- Fokus pada demonstrasi konsep OS
- Interactive tutorials built-in
- Visualization tools untuk process, memory

Option B: Embedded System OS

- Minimal footprint (< 50 MB)
- Fast boot time (< 5 detik)
- Optimized untuk single-purpose application

Option C: Server OS

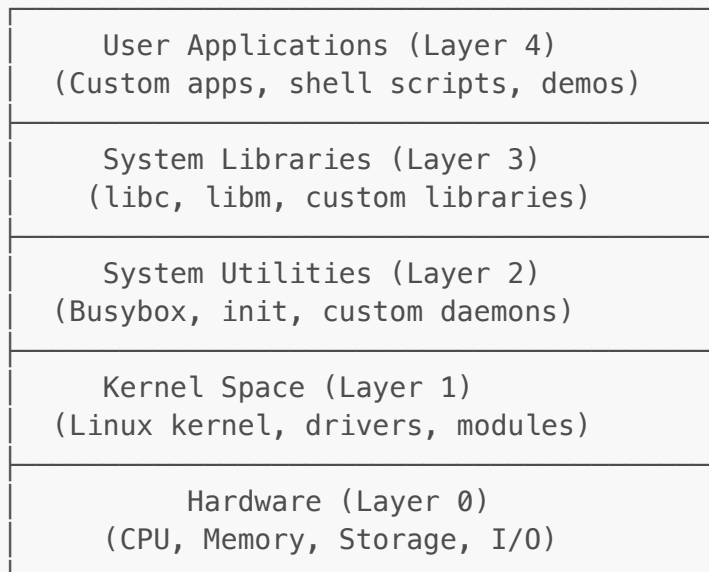
- Network stack optimization
- Multi-user support
- Service management system

Option D: Real-Time OS

- PREEMPT_RT kernel patches
 - Deterministic scheduling
 - Low-latency optimization
-

4. ARSITEKTUR SISTEM

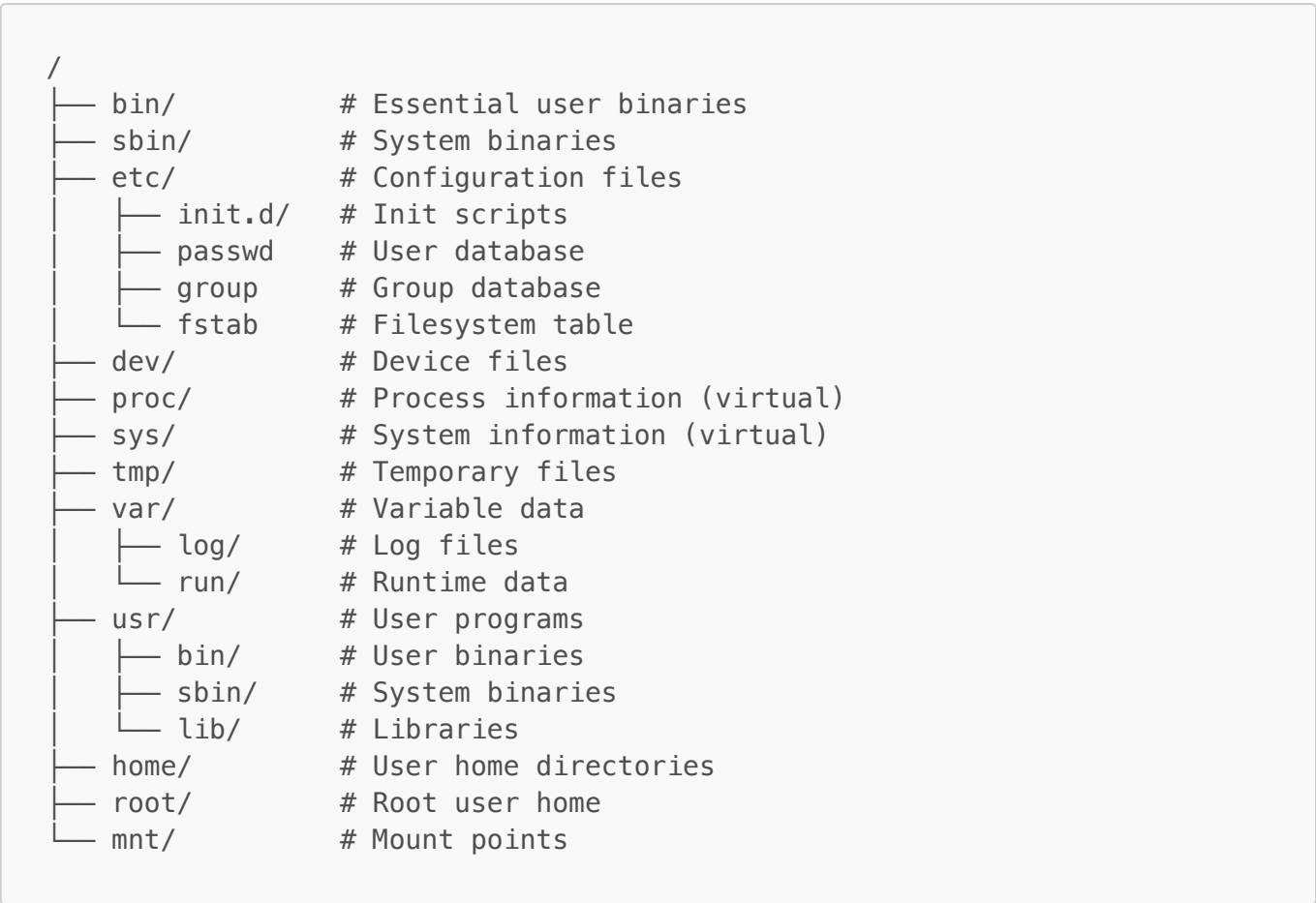
4.1 Layer Architecture



4.2 Boot Process Flow

1. BIOS/UEFI
↓
2. Bootloader (GRUB/SYSLINUX)
↓
3. Kernel Loading
↓
4. Kernel Initialization
 - Hardware detection
 - Memory management init
 - Scheduler init↓
5. Mount root filesystem (initramfs atau disk)
↓
6. Execute /init (PID 1)
↓
7. System Initialization
 - Mount virtual filesystems (/proc, /sys, /dev)
 - Network configuration
 - Start services↓
8. Login prompt / Application launch

4.3 Filesystem Hierarchy



5. TAHAPAN IMPLEMENTASI

5.1 Timeline Proyek

Minggu	Fase	Deliverable
1	Setup & Planning	Proposal dokumen, environment setup
1	Kernel Building	Working kernel image
2	Root Filesystem	Bootable minimal system
2	InitRAMFS & Testing	System boots successfully
3	Feature Implementation	Custom programs, services
3	Optimization	Performance benchmarks
4	Bootloader & Deploy	Bootable ISO/USB image
4	Documentation & Demo	Final presentation

5.2 Milestone Checklist

Milestone 1: Environment Ready (Minggu 1)

- ☐ Host system configured
- ☐ All dependencies installed
- ☐ QEMU testing environment working
- ☐ Git repository initialized

Milestone 2: Kernel Compiled (Minggu 2)

- ☐ Kernel source downloaded
- ☐ Configuration completed
- ☐ Compilation successful
- ☐ Kernel boots in QEMU

Milestone 3: Basic System (Minggu 3)

- ☐ Root filesystem created
- ☐ Busybox installed
- ☐ Init script working
- ☐ Shell accessible

Milestone 4: Full System (Minggu 4)

- ☐ All planned features implemented
- ☐ Custom applications working
- ☐ System stable
- ☐ Basic testing passed

Milestone 5: Production Ready (Minggu 4)

- ☐ Bootloader configured
 - ☐ ISO/USB image created
 - ☐ Documentation complete
 - ☐ Demo video recorded
-

6. PANDUAN TEKNIS DETAIL

6.1 FASE 1: Persiapan Environment

6.1.1 Setup Host System

Install Dependencies (Ubuntu/Debian):

```
#!/bin/bash
# setup-environment.sh

echo "Installing build dependencies..."

sudo apt-get update
sudo apt-get install -y \
    build-essential \
    git \
    wget \
    curl \
    vim \
    libncurses-dev \
    bison \
    flex \
    libssl-dev \
    libelf-dev \
    bc \
    cpio \
    qemu-system-x86 \
    qemu-utils \
    grub-pc-bin \
    xorriso \
    mtools \
    dosfstools \
    genisoimage \
    syslinux \
    isolinux

echo "Creating project directory structure..."
mkdir -p ~/customos/{kernel,rootfs,build,iso,tools,docs}

echo "Environment setup complete!"
```

Install Dependencies (Arch Linux):

```
sudo pacman -S base-devel git wget curl vim ncurses \
    bison flex openssl elfutils bc cpio qemu grub \
    xorriso mtools dosfstools cdrtools syslinux
```


6.1.2 Project Structure

```
# Buat struktur direktori proyek
mkdir -p ~/customos
cd ~/customos

# Struktur lengkap
mkdir -p {
    kernel/src,
    kernel/patches,
    rootfs/{bin,sbin,etc,proc,sys,dev,tmp,var,usr,home,root},
    rootfs/usr/{bin,sbin,lib,share},
    rootfs/etc/{init.d,network},
    build/initramfs,
    iso/isolinux,
    tools/scripts,
    docs,
    src/custom-apps
}

# Initialize git repository
git init
echo "build/" > .gitignore
echo "*.o" >> .gitignore
echo "*.img" >> .gitignore
```

6.1.3 Create Master Build Script

File: tools/scripts/build-all.sh

```
#!/bin/bash
# Master build script

set -e # Exit on error

PROJECT_ROOT="$HOME/customos"
KERNEL_VERSION="6.1.63"
BUSYBOX_VERSION="1.36.0"
NUM_CORES=$(nproc)

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

log_info() {
    echo -e "${GREEN}[INFO]${NC} $1"
}
```

```
log_warn() {
    echo -e "${YELLOW}[WARN]${NC} $1"
}

log_error() {
    echo -e "${RED}[ERROR]${NC} $1"
}

# Download sources
download_sources() {
    log_info "Downloading kernel source..."
    cd "$PROJECT_ROOT/kernel"
    if [ ! -f "linux-${KERNEL_VERSION}.tar.xz" ]; then
        wget
https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-${KERNEL_VERSION}.tar.x
z
        tar xf linux-${KERNEL_VERSION}.tar.xz
    fi

    log_info "Downloading busybox..."
    cd "$PROJECT_ROOT"
    if [ ! -f "busybox-${BUSYBOX_VERSION}.tar.bz2" ]; then
        wget
https://busybox.net/downloads/busybox-${BUSYBOX_VERSION}.tar.bz2
        tar xf busybox-${BUSYBOX_VERSION}.tar.bz2
    fi
}

# Build kernel
build_kernel() {
    log_info "Building Linux kernel..."
    cd "$PROJECT_ROOT/kernel/linux-${KERNEL_VERSION}"

    if [ ! -f .config ]; then
        log_warn "No kernel config found, using defconfig..."
        make defconfig
    fi

    make -j${NUM_CORES}
    log_info "Kernel build complete!"
}

# Build busybox
build_busybox() {
    log_info "Building Busybox..."
    cd "$PROJECT_ROOT/busybox-${BUSYBOX_VERSION}"

    if [ ! -f .config ]; then
        make defconfig
        # Enable static build
        sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
    fi

    make -j${NUM_CORES}
```

```

    make CONFIG_PREFIX="$PROJECT_ROOT/rootfs" install
    log_info "Busybox build complete!"
}

# Create initramfs
create_initramfs() {
    log_info "Creating initramfs..."
    cd "$PROJECT_ROOT/rootfs"
    find . | cpio -H newc -o | gzip >
"$PROJECT_ROOT/build/initramfs.cpio.gz"
    log_info "Initramfs created!"
}

# Test in QEMU
test_qemu() {
    log_info "Testing in QEMU..."
    qemu-system-x86_64 \
        -kernel
"$PROJECT_ROOT/kernel/linux-${KERNEL_VERSION}/arch/x86_64/boot/bzImage" \
        -initrd "$PROJECT_ROOT/build/initramfs.cpio.gz" \
        -append "console=ttyS0" \
        -nographic \
        -m 512M
}

# Main execution
main() {
    log_info "Starting CustomOS build process..."

    download_sources
    build_kernel
    build_busybox
    create_initramfs

    log_info "Build complete! Run 'test_qemu' to test the system."
}

# Parse command line arguments
case "$1" in
    kernel)
        build_kernel
        ;;
    busybox)
        build_busybox
        ;;
    initramfs)
        create_initramfs
        ;;
    test)
        test_qemu
        ;;
    all)
        main
        ;;

```

```
*)
    echo "Usage: $0 {kernel|busybox|initramfs|test|all}"
    exit 1
;;
esac
```

6.2 FASE 2: Kernel Configuration & Compilation

6.2.1 Kernel Configuration Strategy

Minimal Configuration Approach:

```
cd ~/customos/kernel/linux-6.1.63

# Mulai dengan konfigurasi minimal
make allnoconfig

# Jalankan menuconfig untuk enable fitur yang diperlukan
make menuconfig
```

Required Kernel Options:

```
General setup --->
[*] Initial RAM filesystem and RAM disk (initramfs/initrd) support
[*] Configure standard kernel features (expert users) --->
    [*] Enable support for printk
[*] System V IPC
[*] POSIX Message Queues

[*] 64-bit kernel (untuk x86_64)

Processor type and features --->
    [*] Symmetric multi-processing support
    Processor family (Generic x86-64)

[*] Enable loadable module support --->
    [*] Module unloading
    [*] Forced module unloading

[*] Enable the block layer --->
    [*] Block layer SG support v4

Device Drivers --->
    Generic Driver Options --->
        [*] Maintain a devtmpfs filesystem to mount at /dev
        [*] Automount devtmpfs at /dev
```

```

Character devices ---->
    [*] Enable TTY
Serial drivers ---->
    [*] 8250/16550 and compatible serial support
    [*] Console on 8250/16550 and compatible serial port

[*] Network device support ---->
    [*] Ethernet driver support ---->
        [*] Intel devices
        [*] Realtek devices

SCSI device support ---->
    [*] SCSI device support
    [*] SCSI disk support

[*] Serial ATA and Parallel ATA drivers ---->
    [*] AHCI SATA support
    [*] Generic ATA support

File systems ---->
    [*] Second extended fs support
    [*] The Extended 4 (ext4) filesystem
Pseudo filesystems ---->
    [*] /proc file system support
    [*] sysfs file system support
    [*] Tmpfs virtual memory file system
-* Native language support ---->
    <*> Codepage 437
    <*> NLS UTF-8

Executable file formats ---->
    [*] Kernel support for ELF binaries
    [*] Kernel support for scripts starting with #!

Networking support ---->
    [*] Networking support
Networking options ---->
    [*] TCP/IP networking
    [*] IP: kernel level autoconfiguration

```

6.2.2 Custom Kernel Configuration File

File: kernel/configs/minimal-config

```

#!/bin/bash
# Script untuk apply minimal configuration

cd ~/customos/kernel/linux-6.1.63

# Start with allnoconfig
make allnoconfig

```

```
# Enable required options using scripts/config
scripts/config --enable 64BIT
scripts/config --enable SMP
scripts/config --enable BLK_DEV_INITRD
scripts/config --enable RD_GZIP
scripts/config --enable DEVTMPFS
scripts/config --enable DEVTMPFS_MOUNT
scripts/config --enable TTY
scripts/config --enable SERIAL_8250
scripts/config --enable SERIAL_8250_CONSOLE
scripts/config --enable EXT4_FS
scripts/config --enable PROC_FS
scripts/config --enable SYSFS
scripts/config --enable TMPFS

# Network support
scripts/config --enable NET
scripts/config --enable INET
scripts/config --enable NETDEVICES
scripts/config --enable ETHERNET

# Block devices
scripts/config --enable BLK_DEV
scripts/config --enable ATA
scripts/config --enable ATA_PIIX
scripts/config --enable SCSI
scripts/config --enable BLK_DEV_SD

# Update config
make olddefconfig

echo "Minimal kernel configuration applied!"
```

6.2.3 Kernel Compilation

```
# Compile kernel
cd ~/customos/kernel/linux-6.1.63

# Clean previous builds (optional)
make mrproper

# Apply configuration
make defconfig # atau use custom config

# Compile dengan parallel jobs
make -j$(nproc)

# Hasil kompilasi:
# - arch/x86/boot/bzImage (compressed kernel image)
# - vmlinux (uncompressed kernel image)
```

```
# Install modules (jika ada)
make modules_install INSTALL_MOD_PATH=~/.customos/rootfs

# Check kernel size
ls -lh arch/x86/boot/bzImage
```

6.2.4 Kernel Optimization Tips

Untuk Ukuran Minimal:

```
# Disable debugging
scripts/config --disable DEBUG_KERNEL
scripts/config --disable DEBUG_INFO

# Disable modules jika tidak perlu
scripts/config --disable MODULES

# Use kernel compression
scripts/config --enable KERNEL_XZ # atau LZMA, tergantung size vs speed
```

Untuk Performance:

```
# Enable various optimizations
scripts/config --enable CC_OPTIMIZE_FOR_PERFORMANCE
scripts/config --disable CC_OPTIMIZE_FOR_SIZE
scripts/config --enable PREEMPT_VOLUNTARY
```

6.3 FASE 3: Root Filesystem Construction

6.3.1 Busybox Configuration

```
cd ~/.customos/busybox-1.36.0

# Default configuration
make defconfig

# Customize via menuconfig
make menuconfig
```

Important Busybox Settings:

```
Settings --->
[*] Build static binary (no shared libs)
```

```
[*] Build with Large File Support (LFS)
(/bin/sh) Path to sh
[*] Support --install [-s] to install applet links at runtime

Coreutils --->
  [*] ls, cp, mv, rm, mkdir, etc.

Editors --->
  [*] vi

Finding Utilities --->
  [*] find
  [*] grep

Init Utilities --->
  [*] init
  [*] poweroff, halt, and reboot

Shells --->
  [*] ash (most compatible)

System Logging Utilities --->
  [*] syslogd
  [*] klogd

Networking Utilities --->
  [*] ifconfig
  [*] ping
  [*] wget
```

Compile dan Install:

```
# Compile
make -j$(nproc)

# Install ke rootfs
make CONFIG_PREFIX=~/customos/rootfs install

# Verifikasi
ls -la ~/customos/rootfs/bin/busybox
```

6.3.2 Init System Implementation

File: rootfs/init

```
#!/bin/sh
# CustomOS Init Script - PID 1

# Mount virtual filesystems
```



```
# Start getty on console
exec /sbin/getty -n -l /bin/sh 38400 tty1
```

Buat executable:

```
chmod +x ~/customos/rootfs/init
```

6.3.3 Essential Configuration Files

File: rootfs/etc/passwd

```
root:x:0:0:root:/root:/bin/sh
daemon:x:1:1:daemon:/usr/sbin:/bin/false
bin:x:2:2:bin:/bin:/bin/false
sys:x:3:3:sys:/dev:/bin/false
nobody:x:65534:65534:nobody:/nonexistent:/bin/false
```

File: rootfs/etc/group

```
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
tty:x:5:
disk:x:6:
wheel:x:10:root
nogroup:x:65534:
```

File: rootfs/etc/shadow

```
root::19000:0:99999:7:::
daemon*:19000:0:99999:7:::
bin*:19000:0:99999:7:::
sys*:19000:0:99999:7:::
nobody*:19000:0:99999:7:::
```

File: rootfs/etc/fstab

# <file system>	<mount point>	<type>	<options>	<dump>	<pass>
proc	/proc	proc	defaults	0	0
sysfs	/sys	sysfs	defaults	0	0

devpts	/dev/pts	devpts	gid=5,mode=620	0	0
tmpfs	/tmp	tmpfs	defaults	0	0
tmpfs	/run	tmpfs	defaults	0	0

File: rootfs/etc/inittab

```
# /etc/inittab
::sysinit:/etc/init.d/rcS
::respawn:/sbin/getty -L tty1 115200 vt100
::askfirst:/bin/sh
::ctrlaltdel:/sbin/reboot
::shutdown:/bin/umount -a -r
::restart:/sbin/init
```

File: rootfs/etc/profile

```
# System-wide profile

export PATH=/bin:/sbin:/usr/bin:/usr/sbin
export HOME=/root
export PS1='[\u@\h \W]\$ '

# Set terminal
export TERM=linux

# Aliases
alias ll='ls -la'
alias ..='cd ..'

echo "Welcome to CustomOS!"
```

File: rootfs/etc/hostname

```
customos
```

File: rootfs/etc/hosts

```
127.0.0.1    localhost
127.0.1.1    customos
```

6.3.4 Network Configuration**File: rootfs/etc/network/interfaces**

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
```

File: rootfs/etc/init.d/network

```
#!/bin/sh
# Network initialization script

case "$1" in
    start)
        echo "Starting network..."
        ifconfig lo 127.0.0.1 up

        # Try DHCP on eth0
        if [ -x /sbin/udhcpc ]; then
            udhcpc -i eth0 -s /etc/udhcpc/default.script &
        fi
        ;;
    stop)
        echo "Stopping network..."
        ifconfig eth0 down
        ;;
    *)
        echo "Usage: $0 {start|stop}"
        exit 1
        ;;
esac
```

```
chmod +x ~/customos/rootfs/etc/init.d/network
```

6.4 FASE 4: Custom Applications

6.4.1 Process Monitor

File: src/custom-apps/procmon.c

```
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
```

```

#define PROC_DIR "/proc"
#define MAX_PATH 256
#define MAX_LINE 1024

typedef struct {
    int pid;
    char name[256];
    char state;
    unsigned long vsize;
    long rss;
} ProcessInfo;

int is_numeric(const char *str) {
    while (*str) {
        if (!isdigit(*str)) return 0;
        str++;
    }
    return 1;
}

int read_process_info(int pid, ProcessInfo *info) {
    char path[MAX_PATH];
    char line[MAX_LINE];
    FILE *fp;

    snprintf(path, sizeof(path), "%s/%d/stat", PROC_DIR, pid);
    fp = fopen(path, "r");
    if (!fp) return -1;

    if (fgets(line, sizeof(line), fp) == NULL) {
        fclose(fp);
        return -1;
    }

    sscanf(line, "%d %s %c %*d %*d %*d %*d %*d %*u %*u %*u %*u %*u "
               "%*u %*u %*d %*d %*d %*d %*d %*d %*u %lu %ld",
           &info->pid, info->name, &info->state,
           &info->vsize, &info->rss);

    fclose(fp);
    return 0;
}

void print_header() {
    printf("\n");
    printf("%-8s %-20s %-6s %-10s %-10s\n",
           "PID", "NAME", "STATE", "VSIZE", "RSS");
    printf("%-8s %-20s %-6s %-10s %-10s\n",
           "-----", "-----", "-----",
           "-----", "-----");
}

void print_process(ProcessInfo *info) {
    printf("%-8d %-20s %-6c %-10lu %-10ld\n",

```

```

        info->pid, info->name, info->state,
        info->vsize / 1024, info->rss * 4); // Convert to KB
    }

int main(int argc, char *argv[]) {
    DIR *dir;
    struct dirent *entry;
    ProcessInfo info;
    int continuous = 0;

    if (argc > 1 && strcmp(argv[1], "-c") == 0) {
        continuous = 1;
    }

    do {
        system("clear");
        printf("=== CustomOS Process Monitor ===\n");

        dir = opendir(PROC_DIR);
        if (!dir) {
            perror("Cannot open /proc");
            return 1;
        }

        print_header();

        while ((entry = readdir(dir)) != NULL) {
            if (is_numeric(entry->d_name)) {
                int pid = atoi(entry->d_name);
                if (read_process_info(pid, &info) == 0) {
                    print_process(&info);
                }
            }
        }

        closedir(dir);

        if (continuous) {
            printf("\nPress Ctrl+C to exit...\n");
            sleep(2);
        }
    } while (continuous);

    return 0;
}

```

Compile:

```
gcc -o ~/customos/rootfs/usr/bin/procmon ~/customos/src/custom-
apps/procmon.c
```

6.4.2 Memory Information Tool

File: src/custom-apps/meminfo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MEMINFO_FILE "/proc/meminfo"

typedef struct {
    unsigned long total;
    unsigned long free;
    unsigned long available;
    unsigned long buffers;
    unsigned long cached;
    unsigned long swap_total;
    unsigned long swap_free;
} MemInfo;

int parse_meminfo(MemInfo *info) {
    FILE *fp;
    char line[256];
    char key[64];
    unsigned long value;

    fp = fopen(MEMINFO_FILE, "r");
    if (!fp) {
        perror("Cannot open /proc/meminfo");
        return -1;
    }

    while (fgets(line, sizeof(line), fp)) {
        if (sscanf(line, "%s %lu", key, &value) == 2) {
            if (strcmp(key, "MemTotal:") == 0)
                info->total = value;
            else if (strcmp(key, "MemFree:") == 0)
                info->free = value;
            else if (strcmp(key, "MemAvailable:") == 0)
                info->available = value;
            else if (strcmp(key, "Buffers:") == 0)
                info->buffers = value;
            else if (strcmp(key, "Cached:") == 0)
                info->cached = value;
            else if (strcmp(key, "SwapTotal:") == 0)
                info->swap_total = value;
            else if (strcmp(key, "SwapFree:") == 0)
                info->swap_free = value;
        }
    }

    fclose(fp);
}
```

```
        return 0;
    }

void print_memory_bar(const char *label, unsigned long used, unsigned long
total) {
    int bar_length = 50;
    int filled = (int)((double)used / total * bar_length);

    printf("%-15s [", label);
    for (int i = 0; i < bar_length; i++) {
        if (i < filled)
            printf("#");
        else
            printf("-");
    }
    printf("] %.1f%%\n", (double)used / total * 100);
}

void print_size(const char *label, unsigned long kb) {
    if (kb < 1024) {
        printf("%-20s: %lu KB\n", label, kb);
    } else if (kb < 1024 * 1024) {
        printf("%-20s: %.2f MB\n", label, kb / 1024.0);
    } else {
        printf("%-20s: %.2f GB\n", label, kb / 1024.0 / 1024.0);
    }
}

int main() {
    MemInfo info = {0};

    if (parse_meminfo(&info) != 0) {
        return 1;
    }

    printf("\n=== Memory Information ===\n\n");

    print_size("Total Memory", info.total);
    print_size("Free Memory", info.free);
    print_size("Available Memory", info.available);
    print_size("Buffers", info.buffers);
    print_size("Cached", info.cached);

    unsigned long used = info.total - info.available;
    print_size("Used Memory", used);

    printf("\n");
    print_memory_bar("Memory Usage", used, info.total);

    if (info.swap_total > 0) {
        printf("\n");
        print_size("Swap Total", info.swap_total);
        print_size("Swap Free", info.swap_free);
        unsigned long swap_used = info.swap_total - info.swap_free;
```



```

        print_memory_bar("Swap Usage", swap_used, info.swap_total);
    }

    printf("\n");
    return 0;
}

```

Compile:

```

gcc -o ~/customos/rootfs/usr/bin/meminfo ~/customos/src/custom-
apps/meminfo.c

```

6.4.3 Simple File Manager**File: src/custom-apps/filemgr.c**

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <pwd.h>
#include <grp.h>

void print_permissions(mode_t mode) {
    printf("%c", S_ISDIR(mode) ? 'd' : '-');
    printf("%c", mode & S_IRUSR ? 'r' : '-');
    printf("%c", mode & S_IWUSR ? 'w' : '-');
    printf("%c", mode & S_IXUSR ? 'x' : '-');
    printf("%c", mode & S_IRGRP ? 'r' : '-');
    printf("%c", mode & S_IWGRP ? 'w' : '-');
    printf("%c", mode & S_IXGRP ? 'x' : '-');
    printf("%c", mode & S_IROTH ? 'r' : '-');
    printf("%c", mode & S_IWOTH ? 'w' : '-');
    printf("%c", mode & S_IXOTH ? 'x' : '-');
}

void list_directory(const char *path, int detailed) {
    DIR *dir;
    struct dirent *entry;
    struct stat st;
    char fullpath[1024];

    dir = opendir(path);
    if (!dir) {
        perror("opendir");
        return;
    }
}

```

```

printf("\nDirectory: %s\n\n", path);

if (detailed) {
    printf("%-11s %-8s %-8s %-10s %-20s %s\n",
        "Permissions", "Owner", "Group", "Size", "Modified",
        "Name");
    printf("%-11s %-8s %-8s %-10s %-20s %s\n",
        "-----", "-----", "-----", "-----",
        "-----", "-----");
}

while ((entry = readdir(dir)) != NULL) {
    snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry-
>d_name);

    if (stat(fullpath, &st) == 0) {
        if (detailed) {
            print_permissions(st.st_mode);

            struct passwd *pw = getpwuid(st.st_uid);
            struct group *gr = getgrgid(st.st_gid);

            printf(" %-8s", pw ? pw->pw_name : "?");
            printf(" %-8s", gr ? gr->gr_name : "?");
            printf(" %-10ld", (long)st.st_size);

            char timebuf[64];
            strftime(timebuf, sizeof(timebuf), "%Y-%m-%d %H:%M:%S",
                localtime(&st.st_mtime));
            printf(" %-20s", timebuf);

            printf(" %s", entry->d_name);
            if (S_ISDIR(st.st_mode))
                printf("/");
            printf("\n");
        } else {
            printf("%s", entry->d_name);
            if (S_ISDIR(st.st_mode))
                printf("/");
            printf("\n");
        }
    }
}

closedir(dir);
}

int main(int argc, char *argv[]) {
    const char *path = ".";
    int detailed = 0;

    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-l") == 0) {

```

```

        detailed = 1;
    } else {
        path = argv[i];
    }
}

list_directory(path, detailed);
return 0;
}

```

Compile:

```
gcc -o ~/customos/rootfs/usr/bin/filemgr ~/customos/src/custom-
apps/filemgr.c
```

6.4.4 System Information Dashboard**File: src/custom-apps/sysinfo.c**

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/utsname.h>
#include <sys/sysinfo.h>
#include <time.h>

void print_system_info() {
    struct utsname uts;
    struct sysinfo si;

    if (uname(&uts) != 0 || sysinfo(&si) != 0) {
        perror("Failed to get system info");
        return;
    }

    printf("\n");
    printf("CustomOS System Information\n");
    printf("Operating System : %s\n", uts.sysname);
    printf("Kernel Version   : %s\n", uts.release);
    printf("Hostname         : %s\n", uts.nodename);
    printf("Architecture     : %s\n", uts.machine);

    long uptime_days = si.uptime / 86400;
    long uptime_hours = (si.uptime % 86400) / 3600;
    long uptime_mins = (si.uptime % 3600) / 60;

```

```

printf("Uptime          : %ld days, %ld hours, %ld minutes\n",
      uptime_days, uptime_hours, uptime_mins);

printf("\n--- Memory Information ---\n");
printf("Total RAM          : %.2f MB\n", si.totalram / 1024.0 / 1024.0);
printf("Free RAM           : %.2f MB\n", si.freeram / 1024.0 / 1024.0);
printf("Shared RAM        : %.2f MB\n", si.sharedram / 1024.0 /
1024.0);
printf("Buffer RAM         : %.2f MB\n", si.bufferram / 1024.0 /
1024.0);

printf("\n--- Process Information ---\n");
printf("Total Processes   : %d\n", si.procs);

printf("\n--- Load Average ---\n");
printf("1 min   : %.2f\n", si.loads[0] / 65536.0);
printf("5 min   : %.2f\n", si.loads[1] / 65536.0);
printf("15 min  : %.2f\n", si.loads[2] / 65536.0);

printf("\n");
}

int main() {
    print_system_info();
    return 0;
}

```

Compile:

```
gcc -o ~/customos/rootfs/usr/bin/sysinfo ~/customos/src/custom-
apps/sysinfo.c
```

6.4.5 Makefile untuk Custom Apps**File: src/custom-apps/Makefile**

```

CC = gcc
CFLAGS = -Wall -O2
DESTDIR = ../../rootfs/usr/bin

TARGETS = procmon meminfo filemgr sysinfo

all: $(TARGETS)

procmon: procmon.c
    $(CC) $(CFLAGS) -o $@ $<

meminfo: meminfo.c
    $(CC) $(CFLAGS) -o $@ $<

```

```

filemgr: filemgr.c
    $(CC) $(CFLAGS) -o $@ $<

sysinfo: sysinfo.c
    $(CC) $(CFLAGS) -o $@ $<

install: all
    mkdir -p $(DESTDIR)
    cp $(TARGETS) $(DESTDIR)/
    chmod +x $(DESTDIR)/*

clean:
    rm -f $(TARGETS) *.o

.PHONY: all install clean

```

6.5 FASE 5: InitRAMFS Creation & Testing

6.5.1 Create InitRAMFS

File: tools/scripts/create-initramfs.sh

```

#!/bin/bash

set -e

PROJECT_ROOT="$HOME/customos"
ROOTFS="$PROJECT_ROOT/rootfs"
OUTPUT="$PROJECT_ROOT/build/initramfs.cpio.gz"

echo "Creating initramfs from $ROOTFS..."

# Ensure init is executable
chmod +x "$ROOTFS/init"

# Create device nodes if they don't exist
sudo mknod -m 666 "$ROOTFS/dev/null" c 1 3 2>/dev/null || true
sudo mknod -m 600 "$ROOTFS/dev/console" c 5 1 2>/dev/null || true
sudo mknod -m 666 "$ROOTFS/dev/tty" c 5 0 2>/dev/null || true

# Set proper permissions
chmod 755 "$ROOTFS"
chmod 1777 "$ROOTFS/tmp"

# Create initramfs
cd "$ROOTFS"
find . -print0 | cpio --null -ov --format=newc | gzip -9 > "$OUTPUT"

echo "Initramfs created: $OUTPUT"
echo "Size: $(du -h $OUTPUT | cut -f1)"

```

6.5.2 QEMU Testing Scripts

File: tools/scripts/test-qemu.sh

```
#!/bin/bash

PROJECT_ROOT="$HOME/customos"
KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
INITRD="$PROJECT_ROOT/build/initramfs.cpio.gz"

# Check if files exist
if [ ! -f "$KERNEL" ]; then
    echo "Error: Kernel not found at $KERNEL"
    exit 1
fi

if [ ! -f "$INITRD" ]; then
    echo "Error: InitRAMFS not found at $INITRD"
    exit 1
fi

echo "Starting CustomOS in QEMU..."
echo "Kernel: $KERNEL"
echo "InitRD: $INITRD"
echo ""

# Basic QEMU test
qemu-system-x86_64 \
    -kernel "$KERNEL" \
    -initrd "$INITRD" \
    -append "console=ttyS0 init=/init" \
    -nographic \
    -m 512M \
    -smp 2

# To exit QEMU: Ctrl+A, then X
```

File: tools/scripts/test-qemu-gui.sh

```
#!/bin/bash

PROJECT_ROOT="$HOME/customos"
KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
INITRD="$PROJECT_ROOT/build/initramfs.cpio.gz"

echo "Starting CustomOS in QEMU (GUI mode)..."

qemu-system-x86_64 \
    -kernel "$KERNEL" \
```

```
-initrd "$INITRD" \
-append "console=tty0 init=/init" \
-m 512M \
-smp 2 \
-enable-kvm
```

File: tools/scripts/test-qemu-network.sh

```
#!/bin/bash

PROJECT_ROOT="$HOME/customos"
KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
INITRD="$PROJECT_ROOT/build/initramfs.cpio.gz"

echo "Starting CustomOS with networking..."

qemu-system-x86_64 \
    -kernel "$KERNEL" \
    -initrd "$INITRD" \
    -append "console=ttyS0 init=/init ip=dhcp" \
    -nographic \
    -m 512M \
    -netdev user,id=net0 \
    -device e1000,netdev=net0
```

6.6 FASE 6: Bootloader Configuration

6.6.1 Create Bootable Disk Image

File: tools/scripts/create-bootable-disk.sh

```
#!/bin/bash

set -e

PROJECT_ROOT="$HOME/customos"
DISK_IMAGE="$PROJECT_ROOT/build/customos.img"
MOUNT_POINT="/tmp/customos-mount"

KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
ROOTFS="$PROJECT_ROOT/rootfs"

echo "Creating bootable disk image..."

# Create empty disk image (1GB)
dd if=/dev/zero of="$DISK_IMAGE" bs=1M count=1024

# Create partition table
parted "$DISK_IMAGE" mklabel msdos
```

```
parted "$DISK_IMAGE" mkpart primary ext4 1MiB 100%
parted "$DISK_IMAGE" set 1 boot on

# Setup loop device
LOOP_DEVICE=$(sudo losetup -f -P --show "$DISK_IMAGE")
echo "Loop device: $LOOP_DEVICE"

# Format partition
sudo mkfs.ext4 "${LOOP_DEVICE}p1"

# Mount partition
sudo mkdir -p "$MOUNT_POINT"
sudo mount "${LOOP_DEVICE}p1" "$MOUNT_POINT"

# Copy root filesystem
echo "Copying root filesystem..."
sudo cp -a "$ROOTFS"/* "$MOUNT_POINT"/

# Create boot directory
sudo mkdir -p "$MOUNT_POINT/boot/grub"

# Copy kernel
sudo cp "$KERNEL" "$MOUNT_POINT/boot/vmlinuz"

# Install GRUB
echo "Installing GRUB..."
sudo grub-install --target=i386-pc --boot-directory="$MOUNT_POINT/boot"
"$LOOP_DEVICE"

# Create GRUB configuration
cat << 'EOF' | sudo tee "$MOUNT_POINT/boot/grub/grub.cfg"
set timeout=5
set default=0

menuentry "CustomOS" {
    linux /boot/vmlinuz root=/dev/sda1 ro quiet
}

menuentry "CustomOS (verbose)" {
    linux /boot/vmlinuz root=/dev/sda1 ro
}

menuentry "CustomOS (recovery)" {
    linux /boot/vmlinuz root=/dev/sda1 ro single
}
EOF

# Cleanup
sudo umount "$MOUNT_POINT"
sudo losetup -d "$LOOP_DEVICE"
sudo rmdir "$MOUNT_POINT"

echo "Bootable disk image created: $DISK_IMAGE"
echo "Test with: qemu-system-x86_64 -hda $DISK_IMAGE -m 512M"
```


6.6.2 Create Bootable ISO

File: tools/scripts/create-iso.sh

```
#!/bin/bash

set -e

PROJECT_ROOT="$HOME/customos"
ISO_DIR="$PROJECT_ROOT/iso"
ISO_OUTPUT="$PROJECT_ROOT/build/customos.iso"

KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
INITRD="$PROJECT_ROOT/build/initramfs.cpio.gz"

echo "Creating bootable ISO..."

# Clean and create ISO directory structure
rm -rf "$ISO_DIR"
mkdir -p "$ISO_DIR"/{boot/{grub,isolinux},live}

# Copy kernel and initrd
cp "$KERNEL" "$ISO_DIR/live/vmlinuz"
cp "$INITRD" "$ISO_DIR/live/initrd.gz"

# Copy isolinux files
cp /usr/lib/ISOLINUX/isolinux.bin "$ISO_DIR/boot/isolinux/"
cp /usr/lib/syslinux/modules/bios/ldlinux.c32 "$ISO_DIR/boot/isolinux/"
cp /usr/lib/syslinux/modules/bios/menu.c32 "$ISO_DIR/boot/isolinux/"
cp /usr/lib/syslinux/modules/bios/libutil.c32 "$ISO_DIR/boot/isolinux/"

# Create isolinux configuration
cat << 'EOF' > "$ISO_DIR/boot/isolinux/isolinux.cfg"
DEFAULT menu.c32
PROMPT 0
TIMEOUT 50

MENU TITLE CustomOS Boot Menu

LABEL customos
    MENU LABEL CustomOS
    KERNEL /live/vmlinuz
    APPEND initrd=/live/initrd.gz boot=live

LABEL verbose
    MENU LABEL CustomOS (Verbose)
    KERNEL /live/vmlinuz
    APPEND initrd=/live/initrd.gz boot=live debug

EOF
```

```

# Create GRUB configuration for UEFI
mkdir -p "$ISO_DIR/boot/grub"
cat << 'EOF' > "$ISO_DIR/boot/grub/grub.cfg"
set timeout=5
set default=0

menuentry "CustomOS" {
    linux /live/vmlinuz boot=live
    initrd /live/initrd.gz
}

menuentry "CustomOS (Verbose)" {
    linux /live/vmlinuz boot=live debug
    initrd /live/initrd.gz
}
EOF

# Create ISO
xorriso -as mkisofs \
    -iso-level 3 \
    -full-iso9660-filenames \
    -volid "CUSTOMOS" \
    -eltorito-boot boot/isolinux/isolinux.bin \
    -eltorito-catalog boot/isolinux/boot.cat \
    -no-emul-boot -boot-load-size 4 -boot-info-table \
    -isohybrid-mbr /usr/lib/ISOLINUX/isohdpx.bin \
    -eltorito-alt-boot \
    -e boot/grub/efi.img \
    -no-emul-boot \
    -isohybrid-gpt-basdat \
    -output "$ISO_OUTPUT" \
    "$ISO_DIR" 2>/dev/null || \
    genisoimage -rational-rock -volid "CUSTOMOS" \
    -cache-inodes -joliet -full-iso9660-filenames \
    -b boot/isolinux/isolinux.bin \
    -c boot/isolinux/boot.cat \
    -no-emul-boot -boot-load-size 4 -boot-info-table \
    -output "$ISO_OUTPUT" "$ISO_DIR"

# Make ISO hybrid (bootable from USB)
if command -v isohybrid &> /dev/null; then
    isohybrid "$ISO_OUTPUT"
fi

echo "ISO created: $ISO_OUTPUT"
echo "Size: $(du -h $ISO_OUTPUT | cut -f1)"
echo ""
echo "To test: qemu-system-x86_64 -cdrom $ISO_OUTPUT -m 512M"
echo "To burn to USB: sudo dd if=$ISO_OUTPUT of=/dev/sdX bs=4M"
echo "status=progress"

```

6.7 FASE 7: Optimization & Performance

6.7.1 Size Optimization

File: tools/scripts/optimize-size.sh

```
#!/bin/bash

PROJECT_ROOT="$HOME/customos"
ROOTFS="$PROJECT_ROOT/rootfs"

echo "Optimizing system size..."

# Strip all binaries
echo "Stripping binaries..."
find "$ROOTFS" -type f -executable -exec strip --strip-all {} \;
2>/dev/null || true

# Remove unnecessary files
echo "Removing unnecessary files..."
find "$ROOTFS" -name "*.a" -delete
find "$ROOTFS" -name "*.la" -delete
find "$ROOTFS" -name "*~" -delete

# Compress man pages if any
if [ -d "$ROOTFS/usr/share/man" ]; then
    find "$ROOTFS/usr/share/man" -type f -name "*.?" -exec gzip -9 {} \;
fi

# Remove documentation if minimal build
if [ "$MINIMAL_BUILD" = "yes" ]; then
    rm -rf "$ROOTFS/usr/share/doc"
    rm -rf "$ROOTFS/usr/share/info"
    rm -rf "$ROOTFS/usr/share/man"
fi

echo "Optimization complete!"
du -sh "$ROOTFS"
```

6.7.2 Boot Time Optimization

File: rootfs/etc/init.d/rcS

```
#!/bin/sh
# Fast boot initialization

# Start time tracking
BOOT_START=$(cut -d' ' -f1 /proc/uptime)

# Mount essential filesystems in parallel
```

```

mount -t proc none /proc &
mount -t sysfs none /sys &
mount -t devtmpfs none /dev &
wait

# Initialize random seed for faster boot
[ -f /var/lib/random-seed ] && cat /var/lib/random-seed > /dev/urandom

# Network initialization (background)
/etc/init.d/network start &

# Calculate boot time
BOOT_END=$(cut -d' ' -f1 /proc/uptime)
BOOT_TIME=$(echo "$BOOT_END - $BOOT_START" | bc)
echo "Boot time: ${BOOT_TIME}s" > /var/log/boot-time.log

```

6.7.3 Performance Benchmarking

File: src/custom-apps/benchmark.c

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/resource.h>

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void benchmark_fork() {
    double start = get_time();
    int iterations = 1000;

    for (int i = 0; i < iterations; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            _exit(0);
        }
        wait(NULL);
    }

    double elapsed = get_time() - start;
    printf("Fork/Exec benchmark: %d iterations in %.3fs (%.0f ops/sec)\n",
          iterations, elapsed, iterations / elapsed);
}

void benchmark_memory() {

```

```
double start = get_time();
size_t size = 100 * 1024 * 1024; // 100MB

char *mem = malloc(size);
if (!mem) {
    printf("Memory allocation failed\n");
    return;
}

for (size_t i = 0; i < size; i++) {
    mem[i] = i & 0xFF;
}

free(mem);

double elapsed = get_time() - start;
printf("Memory benchmark: 100MB allocated/written in %.3fs (%.2f
MB/s)\n",
    elapsed, 100 / elapsed);
}

void benchmark_io() {
    double start = get_time();
    FILE *fp = fopen("/tmp/benchmark.dat", "w");
    if (!fp) {
        perror("fopen");
        return;
    }

    char buffer[4096];
    for (int i = 0; i < 10000; i++) {
        fwrite(buffer, 1, sizeof(buffer), fp);
    }

    fclose(fp);
    unlink("/tmp/benchmark.dat");

    double elapsed = get_time() - start;
    double mb = (10000 * 4096) / (1024.0 * 1024.0);
    printf("I/O benchmark: %.2f MB written in %.3fs (%.2f MB/s)\n",
        mb, elapsed, mb / elapsed);
}

int main() {
    printf("\n=== CustomOS Performance Benchmark ===\n\n");

    benchmark_fork();
    benchmark_memory();
    benchmark_io();

    printf("\nBenchmark complete!\n");
    return 0;
}
```

7. TESTING DAN DEBUGGING

7.1 Unit Testing

7.1.1 Kernel Module Test

File: tests/test-kernel.sh

```
#!/bin/bash

echo "=== Kernel Tests ==="

# Test 1: Kernel version
echo -n "Kernel version check... "
if uname -r | grep -q "6.1"; then
    echo "PASS"
else
    echo "FAIL"
fi

# Test 2: Essential filesystems
echo -n "Filesystem check... "
if mount | grep -q "proc" && mount | grep -q "sysfs"; then
    echo "PASS"
else
    echo "FAIL"
fi

# Test 3: Device files
echo -n "Device files check... "
if [ -c /dev/null ] && [ -c /dev/console ]; then
    echo "PASS"
else
    echo "FAIL"
fi
```

7.1.2 System Services Test

File: tests/test-services.sh

```
#!/bin/bash

echo "=== Service Tests ==="

# Test network
echo -n "Network interface... "
if ifconfig lo | grep -q "UP"; then
    echo "PASS"
```

```
else
    echo "FAIL"
fi

# Test custom apps
echo -n "Custom applications... "
if [ -x /usr/bin/procmon ] && [ -x /usr/bin/sysinfo ]; then
    echo "PASS"
else
    echo "FAIL"
fi
```

7.2 Integration Testing

File: tests/integration-test.sh

```
#!/bin/bash

PROJECT_ROOT="$HOME/customos"
KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
INITRD="$PROJECT_ROOT/build/initramfs.cpio.gz"

echo "Running integration tests..."

# Create test script
cat > /tmp/test-script.sh << 'EOF'
#!/bin/sh
echo "=== Integration Test ==="
echo "1. System boot: OK"

echo -n "2. Process management... "
if ps aux > /dev/null 2>&1; then
    echo "OK"
else
    echo "FAIL"
fi

echo -n "3. File system operations... "
if touch /tmp/test && rm /tmp/test; then
    echo "OK"
else
    echo "FAIL"
fi

echo -n "4. Custom apps... "
if /usr/bin/sysinfo > /dev/null 2>&1; then
    echo "OK"
else
    echo "FAIL"
fi
```

```
poweroff -f
EOF

chmod +x /tmp/test-script.sh

# Run test in QEMU with timeout
timeout 30 qemu-system-x86_64 \
    -kernel "$KERNEL" \
    -initrd "$INITRD" \
    -append "console=ttyS0 init=/tmp/test-script.sh" \
    -nographic \
    -m 256M

echo "Integration test complete!"
```

7.3 Debugging Tools

7.3.1 Enable Kernel Debugging

Kernel config untuk debugging:

```
scripts/config --enable DEBUG_KERNEL
scripts/config --enable DEBUG_INFO
scripts/config --enable GDB_SCRIPTS
scripts/config --enable FRAME_POINTER
```

7.3.2 GDB Debugging Setup

File: tools/scripts/debug-qemu.sh

```
#!/bin/bash

PROJECT_ROOT="$HOME/customos"
KERNEL="$PROJECT_ROOT/kernel/linux-6.1.63/arch/x86_64/boot/bzImage"
INITRD="$PROJECT_ROOT/build/initramfs.cpio.gz"

echo "Starting QEMU with GDB server..."

qemu-system-x86_64 \
    -kernel "$KERNEL" \
    -initrd "$INITRD" \
    -append "console=ttyS0 nokaslr" \
    -nographic \
    -s -S \
    -m 512M

# In another terminal:
# gdb vmlinux
# (gdb) target remote :1234
```



```
# (gdb) break start_kernel
# (gdb) continue
```

7.3.3 Logging System

File: rootfs/etc/init.d/logging

```
#!/bin/sh

case "$1" in
    start)
        echo "Starting logging system..."
        mkdir -p /var/log

        # Start syslogd
        if [ -x /sbin/syslogd ]; then
            syslogd -O /var/log/messages
        fi

        # Start klogd
        if [ -x /sbin/klogd ]; then
            klogd
        fi
        ;;
    stop)
        killall syslogd klogd 2>/dev/null
        ;;
esac
```

8. TROUBLESHOOTING

8.1 Common Problems

8.1.1 Kernel Panic

Problem: Kernel panic at boot

```
Kernel panic - not syncing: VFS: Unable to mount root fs
```

Solutions:

1. Check initramfs format:

```
file build/initramfs.cpio.gz
# Should show: gzip compressed data
```

2. Verify init script:

```
# Must be executable
chmod +x rootfs/init

# Check shebang
head -n1 rootfs/init
# Should show: #!/bin/sh
```

3. Check kernel config:

```
grep CONFIG_BLK_DEV_INITRD kernel/linux-6.1.63/.config
# Should be: CONFIG_BLK_DEV_INITRD=y
```

8.1.2 Init Not Found

Problem:

```
Failed to execute /init. Attempting defaults...
Kernel panic - not syncing: No working init found.
```

Solutions:

```
# Verify init location
ls -l rootfs/init

# Check dependencies
ldd rootfs/init
# If dynamic, ensure libraries are in rootfs

# Recreate with correct path
cd rootfs
find . | cpio -H newc -o | gzip > ../build/initramfs.cpio.gz
```

8.1.3 No Console Output

Problem: System boots but no output

Solutions:

1. Add console parameter:

```
qemu-system-x86_64 \  
-append "console=ttyS0,115200 console=tty0"
```

2. Enable serial console in kernel:

```
Device Drivers -> Character devices -> Serial drivers  
[*] 8250/16550 and compatible serial support  
[*] Console on 8250/16550 and compatible serial port
```

8.1.4 Mount Failed

Problem:

```
mount: mounting /dev/sda1 on / failed: Invalid argument
```

Solutions:

1. Check filesystem support in kernel
2. Verify partition format
3. Check fstab syntax

```
# Debug mode  
mount -v /dev/sda1 /mnt  
dmesg | tail
```

8.1.5 Command Not Found

Problem: Busybox commands not working

Solutions:

```
# Verify busybox installation  
ls -l rootfs/bin/busybox  
  
# Reinstall symlinks  
cd rootfs  
./bin/busybox --install -s bin/  
  
# Check PATH  
echo $PATH  
export PATH=/bin:/sbin:/usr/bin:/usr/sbin
```

8.2 Debugging Checklist

Pre-boot Checklist:

- ☐ Kernel compiled successfully
- ☐ initramfs created without errors
- ☐ init script is executable
- ☐ Required kernel options enabled
- ☐ Busybox installed in rootfs

Boot Checklist:

- ☐ Bootloader finds kernel
- ☐ Kernel loads initramfs
- ☐ Init process starts (PID 1)
- ☐ Virtual filesystems mounted
- ☐ Shell accessible

Runtime Checklist:

- ☐ Commands execute correctly
- ☐ File operations work
- ☐ Network configured (if needed)
- ☐ Custom apps functional
- ☐ No error messages in dmesg

8.3 Debugging Commands

```
# Check kernel messages
dmesg | less

# Verify mounts
mount | column -t

# Check processes
ps aux

# Memory status
free -h
cat /proc/meminfo

# Disk usage
df -h

# Network status
ip addr
ip route

# Check init scripts
ls -la /etc/init.d/
```

8.4 Verbose Boot

Enable verbose output:

```
# Remove quiet from kernel parameters
# In GRUB: linux /boot/vmlinuz root=/dev/sda1 ro

# Enable debug messages
echo 8 > /proc/sys/kernel/printk
```

9. KRITERIA PENILAIAN

9.1 Rubrik Penilaian Detail

9.1.1 Fungsionalitas Sistem (30 poin)

Aspek	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Boot Process	Boot < 5 detik, tanpa error	Boot < 10 detik, minor errors	Boot > 10 detik, beberapa error	Gagal boot
Core Functions	Semua fungsi OS berjalan sempurna	Mayoritas fungsi bekerja	Beberapa fungsi bermasalah	Banyak fungsi error
Stability	Tidak ada crash, sangat stabil	Jarang crash, cukup stabil	Kadang crash	Sering crash

Total: 30 poin

9.1.2 Implementasi Konsep OS (25 poin)

Komponen	Poin Max	Kriteria
Kernel Configuration	7	Pemahaman dan optimasi kernel config
Process Management	6	Implementasi demonstrasi process management
Memory Management	6	Monitoring dan optimasi memory
File System	6	Struktur dan management filesystem

Penilaian Detail:

Kernel Configuration (7 poin):

- Minimal config yang efisien: 2 poin
- Optimasi berdasarkan fokus sistem: 2 poin
- Dokumentasi keputusan config: 2 poin
- Penggunaan advanced features: 1 poin

Process Management (6 poin):

- Custom process monitor: 2 poin
- IPC demonstration: 2 poin
- Signal handling implementation: 2 poin

Memory Management (6 poin):

- Memory monitoring tool: 2 poin
- Optimasi memory usage: 2 poin
- Virtual memory demonstration: 2 poin

File System (6 poin):

- Proper FHS structure: 2 poin
- Multiple filesystem types: 2 poin
- Mount management: 2 poin

9.1.3 Kualitas Kode & Desain (20 poin)

Aspek	Poin Max	Kriteria
Code Quality	8	Clean code, formatting, comments
Architecture Design	6	Modular, maintainable, scalable
Best Practices	6	Security, error handling, logging

Code Quality Metrics:

- Consistent naming conventions
- Proper indentation
- Meaningful variable names
- Adequate comments
- No code duplication
- Error handling

Architecture Checklist:

- Modular design
- Separation of concerns
- Reusable components
- Clear dependencies
- Documented interfaces

9.1.4 Dokumentasi (15 poin)

Komponen	Poin Max	Kriteria
Technical Documentation	5	Architecture, design decisions
Build Instructions	4	Complete, reproducible steps

Komponen	Poin Max	Kriteria
User Manual	3	Usage guide, troubleshooting
Code Documentation	3	Inline comments, README files

Documentation Requirements:

- 1. README.md dengan overview project
- 2. BUILD.md dengan step-by-step build instructions
- 3. ARCHITECTURE.md dengan system design
- 4. TROUBLESHOOTING.md dengan common issues
- 5. API documentation untuk custom apps

9.1.5 Inovasi & Optimisasi (10 poin)

Kategori	Poin Max	Contoh
Unique Features	4	GUI, real-time features, custom tools
Performance Optimization	3	Boot time, memory footprint, size
Creative Solutions	3	Innovative problem solving

Bonus Points (maksimal +5):

- Implementasi GUI: +2
- Real-time patches: +2
- Cross-compilation untuk ARM: +2
- Automated testing suite: +1
- CI/CD pipeline: +1

9.2 Deliverables Checklist

9.2.1 Source Code Repository

Required:

```
customos/
├── README.md           # Project overview
├── BUILD.md            # Build instructions
├── ARCHITECTURE.md     # System design
├── TROUBLESHOOTING.md # Common issues
├── LICENSE             # License file
├── .gitignore          # Git ignore rules
├──
├── kernel/
│   ├── configs/        # Kernel configurations
│   └── patches/        # Custom patches
└── rootfs/
```

```

├── init                                # Init script
├── etc/                               # Configuration files
├── usr/                               # User programs
├── src/
│   ├── custom-apps/                  # Custom applications
│   │   ├── Makefile
│   │   ├── procmon.c
│   │   ├── meminfo.c
│   │   └── ...
│   ├── tools/
│   │   ├── scripts/                 # Build automation
│   │   │   ├── build-all.sh
│   │   │   ├── create-iso.sh
│   │   │   └── ...
│   ├── tests/
│   │   ├── test-kernel.sh
│   │   ├── test-services.sh
│   │   └── integration-test.sh
├── docs/
│   ├── design-decisions.md
│   ├── performance-analysis.md
│   └── user-guide.md

```

9.2.2 Bootable Images

Required Files:

1. **customos.iso** - Bootable ISO image
2. **customos.img** - Disk image
3. **initramfs.cpio.gz** - Initial RAM filesystem
4. **bzImage** - Kernel image

Testing Evidence:

- Screenshots dari booting process
- Video demo (3-5 menit)
- Performance benchmarks output

9.2.3 Documentation

Minimum Documentation:

1. README.md:

```
# CustomOS - [Fokus Sistem]
```


Description

Brief description of the OS and its purpose

Features

- List of key features
- Unique selling points

System Requirements

- Host system requirements
- Target hardware specs

Quick Start

Brief instructions to build and run

Authors

Student name and ID

License

License information

2. BUILD.md:

Build Instructions

Prerequisites

Detailed list of dependencies

Step-by-Step Build

1. Environment setup
2. Kernel compilation
3. Rootfs creation
4. Image generation

Testing

How to test in QEMU

Troubleshooting

Common build issues

3. ARCHITECTURE.md:

System Architecture

Overview

High-level architecture diagram

Components

Detailed component descriptions

Boot Process

Step-by-step boot sequence

Design Decisions

Rationale behind key decisions

Performance Considerations

Optimization strategies used

9.2.4 Presentation

Format: 15-20 menit presentasi + 5-10 menit Q&A

Struktur Presentasi:

1. Introduction (2 menit)

- Project overview
- Fokus sistem yang dipilih
- Motivasi

2. Architecture & Design (4 menit)

- System architecture diagram
- Key components
- Design decisions

3. Implementation (5 menit)

- Kernel configuration
- Custom features
- Challenges & solutions

4. Live Demo (4 menit)

- Boot process
- Core functionality
- Custom applications

5. Performance & Evaluation (3 menit)

- Benchmarks
- Optimization results
- Comparison dengan target

6. Lessons Learned (2 menit)

- Key takeaways
- Future improvements

9.3 Grading Scale

A (85-100):

- Sistem berjalan sempurna tanpa error
- Implementasi konsep OS sangat baik
- Kode berkualitas tinggi dengan dokumentasi lengkap
- Menunjukkan inovasi dan optimasi signifikan
- Presentasi profesional dan confident

B (70-84):

- Sistem berjalan dengan baik, minimal error
- Implementasi konsep OS solid
- Kode baik dengan dokumentasi memadai
- Beberapa optimasi dilakukan
- Presentasi jelas dan terstruktur

C (60-69):

- Sistem berjalan dengan beberapa masalah
- Implementasi konsep OS basic
- Kode functional dengan dokumentasi terbatas
- Optimasi minimal
- Presentasi cukup jelas

D (50-59):

- Sistem berjalan tapi banyak masalah
- Implementasi konsep OS tidak lengkap
- Kode memerlukan improvement
- Dokumentasi sangat terbatas
- Presentasi kurang persiapan

E (<50):

- Sistem tidak berjalan dengan baik
- Implementasi konsep OS sangat terbatas
- Kode berkualitas rendah
- Dokumentasi tidak memadai
- Tidak menunjukkan usaha serius

10. REFERENSI

10.1 Dokumentasi Official

Linux Kernel:

- <https://www.kernel.org/doc/html/latest/>
- <https://www.kernel.org/doc/Documentation/>
- <https://kernelnewbies.org/>

Busybox:

- <https://busybox.net/documentation.html>
- <https://busybox.net/FAQ.html>

Build Systems:

- <https://buildroot.org/docs.html>
- <https://www.yoctoproject.org/docs/>

10.2 Books & Resources

Essential Reading:

1. "Linux From Scratch" - Gerard Beekmans
 - <http://www.linuxfromscratch.org/>
2. "Linux Kernel Development" - Robert Love
 - Understanding kernel internals
3. "Understanding the Linux Kernel" - Daniel P. Bovet & Marco Cesati
 - Deep dive into kernel architecture
4. "Advanced Programming in the UNIX Environment" - W. Richard Stevens
 - System programming fundamentals

Online Courses:

- MIT 6.828: Operating System Engineering
- CS 162: Operating Systems (UC Berkeley)
- Linux Foundation Training

10.3 Tools & Utilities

Development Tools:

gcc/clang	- Compiler
make	- Build automation
git	- Version control
vim/emacs	- Text editors
gdb	- Debugger
strace/ltrace	- System call tracer
valgrind	- Memory debugger

Virtualization:

QEMU	- Hardware emulator
VirtualBox	- Virtual machine
KVM	- Kernel virtual machine

Build Tools:

buildroot	- Embedded Linux builder
crosstool-NG	- Cross-compiler generator
debootstrap	- Debian bootstrap

10.4 Community Resources

Forums & Communities:

- LinuxQuestions.org
- Stack Overflow (Linux tag)
- Reddit: r/linux, r/kernel, r/osdev
- IRC: #kernelnewbies on OFTC

Mailing Lists:

- Linux Kernel Mailing List (LKML)
- Busybox mailing list

10.5 Video Tutorials

- "Building a Linux System from Scratch" - YouTube playlist
- "Linux Kernel Programming" - Udemy courses
- "Operating Systems" - MIT OpenCourseWare

10.6 Sample Projects

GitHub Repositories:

- <https://github.com/torvalds/linux> (Official Linux kernel)
- <https://github.com/mirror/busybox> (Busybox mirror)
- <https://github.com/ivandavidov/minimal> (Minimal Linux Live)
- <https://github.com/0xAX/linux-insides> (Linux kernel internals guide)

10.7 Reference Distributions

Minimal Distributions to Study:

1. Alpine Linux - Minimal, security-focused
2. Tiny Core Linux - Ultra-minimal
3. OpenWrt - Router-focused
4. Buildroot samples - Various embedded configs

APPENDIX A: Quick Reference Commands

Kernel Build

```
cd linux-6.1.63
make defconfig
make menuconfig
make -j$(nproc)
```

Busybox Build

```
cd busybox-1.36.0
make defconfig
sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
make -j$(nproc)
make CONFIG_PREFIX=/path/to/rootfs install
```

Create InitRAMFS

```
cd rootfs
find . | cpio -H newc -o | gzip > ../initramfs.cpio.gz
```

Test in QEMU

```
qemu-system-x86_64 \
  -kernel bzImage \
  -initrd initramfs.cpio.gz \
  -append "console=ttyS0" \
  -nographic -m 512M
```

Create ISO

```
genisoimage -o customos.iso \
  -b isolinux/isolinux.bin \
  -c isolinux/boot.cat \
  -no-emul-boot -boot-load-size 4 \
  -boot-info-table -R -J -v -T iso/
```

APPENDIX B: Troubleshooting Matrix

Symptom	Possible Cause	Solution
Kernel panic: VFS can't mount	Wrong initramfs format	Recreate with cpio + gzip
No init found	Missing/incorrect init	Check /init exists and executable
No output	Console not configured	Add console=ttyS0 to cmdline
Command not found	Busybox not installed	Reinstall with --install -s
Permission denied	Wrong file permissions	chmod +x on scripts
Out of memory	Insufficient RAM	Increase QEMU -m parameter
Network not working	Driver not enabled	Enable network driver in kernel
Slow boot	Too many services	Disable unnecessary init scripts

APPENDIX C: Performance Targets

Minimal System Targets

- **Total Size:** < 50 MB
- **Boot Time:** < 5 seconds
- **RAM Usage:** < 64 MB idle
- **Kernel Size:** < 5 MB compressed

Standard System Targets

- **Total Size:** < 200 MB
- **Boot Time:** < 10 seconds
- **RAM Usage:** < 128 MB idle
- **Kernel Size:** < 10 MB compressed

Full-Featured System

- **Total Size:** < 500 MB
- **Boot Time:** < 15 seconds
- **RAM Usage:** < 256 MB idle
- **Kernel Size:** < 15 MB compressed

APPENDIX D: Security Checklist

Basic Security Measures

- ☐ Root password set (even if empty initially)
- ☐ Proper file permissions (755 for dirs, 644 for files)
- ☐ Executable scripts marked correctly (755)
- ☐ No world-writable files except /tmp
- ☐ Minimal attack surface (only necessary services)

Advanced Security (Optional)

- ☐ SELinux/AppArmor policies
 - ☐ Firewall rules (iptables)
 - ☐ Secure boot configuration
 - ☐ Encrypted filesystems
 - ☐ Kernel hardening options
-

GLOSSARY

Kernel - Core component of the operating system that manages system resources

Initramfs - Initial RAM filesystem, used during boot before mounting root filesystem

Busybox - Single binary containing many common Unix utilities

Root Filesystem - The top-level directory structure containing all system files

System Call - Interface between user programs and kernel

Device Driver - Software that allows the kernel to communicate with hardware

Process - Instance of a running program

Virtual Filesystem (VFS) - Abstraction layer for different filesystem types

Bootloader - Program that loads the operating system kernel

Cross-compilation - Compiling code for a different target architecture